CYBERBULLYING AND TOXIC COMMENT DETECTION

USING NATURAL LANGUAGE PROCESSING (NLP)

Submitted in partial fulfillment of the requirements of
the Mini-Project of Natural Language Processing Lab
for Fourth Year of

Bachelors of Engineering

by

Shaikh Abdullah - 53

Department of Artificial Intelligence & Data Science Engineering
Rizvi College of Engineering

University of Mumbai

2026-2027

CERTIFICATE

This is to certify that the mini-project entitled Cyberbullying and Toxic Comment Detection Using Natural Language Processing (NLP) is a bonafide work of Shaikh Abdullah-53 submitted to the University of Mumbai in partial fulfillment of the requirement for the Mini-Project of Natural Langauge Processing Lab for Fourth Year of the Bachelor of Engineering in “Artificial Intelligence & Data Science”.

Guide:

……..

Prof. Vijay Shankar

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Shaikh Abdullah -231A033

Date: 20-10-2026

ABSTRACT

Online platforms allow people to communicate quickly, but they can also be used to post abusive, insulting, threatening, or harmful comments. Cyberbullying and toxic language can affect users' well-being and make online spaces less safe. Manually reviewing large volumes of comments is time-consuming, so an automated system can help flag potentially harmful text for further review.

This mini-project proposes a Cyberbullying and Toxic Comment Detection system using Natural Language Processing (NLP) and machine learning. The proposed workflow accepts a text comment, preprocesses the text, converts it into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF), and uses a trained classifier such as Logistic Regression, Naive Bayes, or Support Vector Machine (SVM) to predict a class. A simple web interface may be developed using Flask, HTML, CSS, and JavaScript.

The system is intended as a support tool rather than a final judge of a user or a comment. Language can be ambiguous, context-dependent, sarcastic, or culturally specific; therefore, predictions may be incorrect and human review remains important. The final performance of the system will be reported only after testing on a held-out dataset. Accuracy, precision, recall, and F1-score can be used to evaluate the classifier.

Keywords: Natural Language Processing, cyberbullying, toxic comments, text classification, TF-IDF, machine learning, Flask.

INDEX

Sr. No.	Title	Page No.
1	Aim	6
2	Problem Statement	7
3	Brief Theory	8
4	Implementation Explanation	10
5	Application Screenshots	13
6	Results	15
7	Conclusion	16
8	References	17

Chapter 1: AIM

The aim of this project is to develop a system that uses Natural Language Processing and machine learning to identify potentially toxic or cyberbullying-related comments in text. The system will preprocess user-entered comments, represent them as numerical features, and classify them using a trained machine-learning model.

The project also aims to:

- understand and apply common NLP text-preprocessing techniques;
- convert text into numerical representations using TF-IDF;
- train and compare suitable text-classification algorithms;
- provide a simple interface for entering a comment and viewing the predicted class;
- evaluate model performance using appropriate classification metrics; and
- emphasize that automated predictions should support, not replace, human moderation.

Chapter 2: PROBLEM STATEMENT

Social media, discussion forums, and online communities receive large numbers of comments every day. Some comments contain insults, harassment, threats, or other harmful language. Reviewing every comment manually can require considerable time and effort, especially when a platform handles a high volume of user-generated content.

The problem addressed by this project is to classify a given text comment as toxic or non-toxic using NLP and machine-learning methods. The system should process the input consistently and provide a prediction that can help a moderator decide which comments may need closer attention.

Objectives

- Prepare labelled text data for training and testing.
- Clean and normalize comment text without removing useful meaning.
- Extract TF-IDF features from the processed text.
- Train a baseline classifier and optionally compare multiple algorithms.
- Measure performance on data not used to train the model.
- Create a basic web interface if the application component is implemented.

Scope and Limitations

The project focuses on text-based classification. It does not determine a person's intent, diagnose behaviour, or guarantee that a comment is harmful. Short comments, spelling variations, slang, sarcasm, quotations, and context-dependent language can cause errors. The exact supported labels depend on the dataset selected. If the dataset contains only toxic and non-toxic labels, the system should not claim to identify separate categories such as threats or identity-based abuse.

Chapter 3: BRIEF THEORY

3.1 Natural Language Processing

Natural Language Processing (NLP) is a field of artificial intelligence concerned with processing and analyzing human language. In this project, NLP techniques prepare comments for machine learning and help represent text in a form that a classifier can use.

3.2 Text Preprocessing

Preprocessing prepares raw comments for analysis. Typical steps may include converting text to lowercase, removing unnecessary whitespace, normalizing selected characters, and handling URLs or user mentions. Tokenization divides text into smaller units such as words. Stop-word removal and stemming or lemmatization may be tested, but should be applied carefully because negation and informal language can carry important meaning.

3.3 TF-IDF Vectorization

Machine-learning algorithms generally require numerical input. TF-IDF (Term Frequency–Inverse Document Frequency) represents a document using weights for terms. A term receives a higher weight when it is frequent in a comment but not common across the entire collection. The vectorizer must be fitted on training data only and then applied to validation or test data to avoid data leakage.

3.4 Text Classification

A classifier learns patterns from labelled examples and predicts a label for new comments. Candidate models include Multinomial Naive Bayes, Logistic Regression, and Support Vector Machine. The best model should be selected using validation results and the project's evaluation goals, rather than assuming that one algorithm will always perform best.

3.5 Candidate Algorithms

Naive Bayes is a probabilistic method that is often used as a fast baseline for text classification. Logistic Regression estimates class probabilities from input features and can work well with sparse TF-IDF vectors. Support Vector Machine (SVM), particularly a linear SVM, can separate classes in a high-dimensional feature space. Actual performance depends on the dataset, preprocessing choices, and model settings.

3.6 Evaluation Metrics

The classifier should be evaluated using a held-out test set. Accuracy measures the fraction of all predictions that are correct. Precision measures how many comments predicted as toxic are actually toxic. Recall measures how many toxic comments are detected. F1-score is the harmonic mean of precision and recall. Because missing harmful comments and incorrectly flagging harmless comments have different consequences, precision, recall, the confusion matrix, and F1-score should be reviewed together.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP) Recall = TP / (TP + FN)

F1-score = 2 × (Precision × Recall) / (Precision + Recall)

Here, TP = true positives, TN = true negatives, FP = false positives, and FN = false negatives.

3.7 Ethical Considerations

A toxic-comment detector can make mistakes and may perform unevenly across dialects, communities, or writing styles. It should be used to flag content for review, not to make unsupported conclusions about a person. The dataset should be handled responsibly, and examples containing abusive language should be presented only when necessary for evaluation.



Figure: Illustration of harmful online comments and their potential emotional impact.

Chapter 4: IMPLEMENTATION EXPLANATION

The proposed system can be implemented in Python. Scikit-learn can be used for TF-IDF vectorization, model training, and evaluation. NLTK may be used for selected text-processing tasks. If a web application is included, Flask can connect the user interface to the trained model.



Figure: Workflow of the proposed toxic-comment detection system.

Implementation Steps

1. Data collection: select a suitable labelled dataset of comments and review its licence, label definitions, and limitations.
2. Data inspection: check missing values, duplicate records, label distribution, and the format of the comment and target columns.
3. Train-test split: divide the data into training and test sets before fitting the vectorizer or classifier.
4. Preprocessing: apply consistent text-cleaning steps to training and incoming comments.
5. Feature extraction: fit a TF-IDF vectorizer on training text and transform the training and test data.
6. Model training: train a baseline classifier such as Logistic Regression or Multinomial Naive Bayes; compare SVM if time and resources permit.
7. Evaluation: calculate accuracy, precision, recall, F1-score, and a confusion matrix on the held-out test set.
8. Application integration: load the fitted vectorizer and model in the Flask application, accept a comment, and display the predicted class.
9. Testing: check normal, toxic, empty, very short, and unusual inputs; document errors and limitations.

Tools and Technologies

Programming language: Python. NLP and machine learning: Scikit-learn and optionally NLTK. Feature representation: TF-IDF. Candidate algorithms: Naive Bayes, Logistic Regression, and SVM. Optional interface: Flask, HTML, CSS, and JavaScript.

System Workflow

10. User enters a comment.
11. Application validates the input.
12. Text is processed using the same preprocessing pipeline used during training.
13. TF-IDF transforms the comment into numerical features.
14. The trained classifier predicts the available class label.
15. The interface displays the prediction and a caution that the result may be incorrect.

Screenshot Placeholders

Replace each placeholder below with a genuine screenshot from your completed application. Do not use illustrative or fabricated screenshots as evidence of implementation.

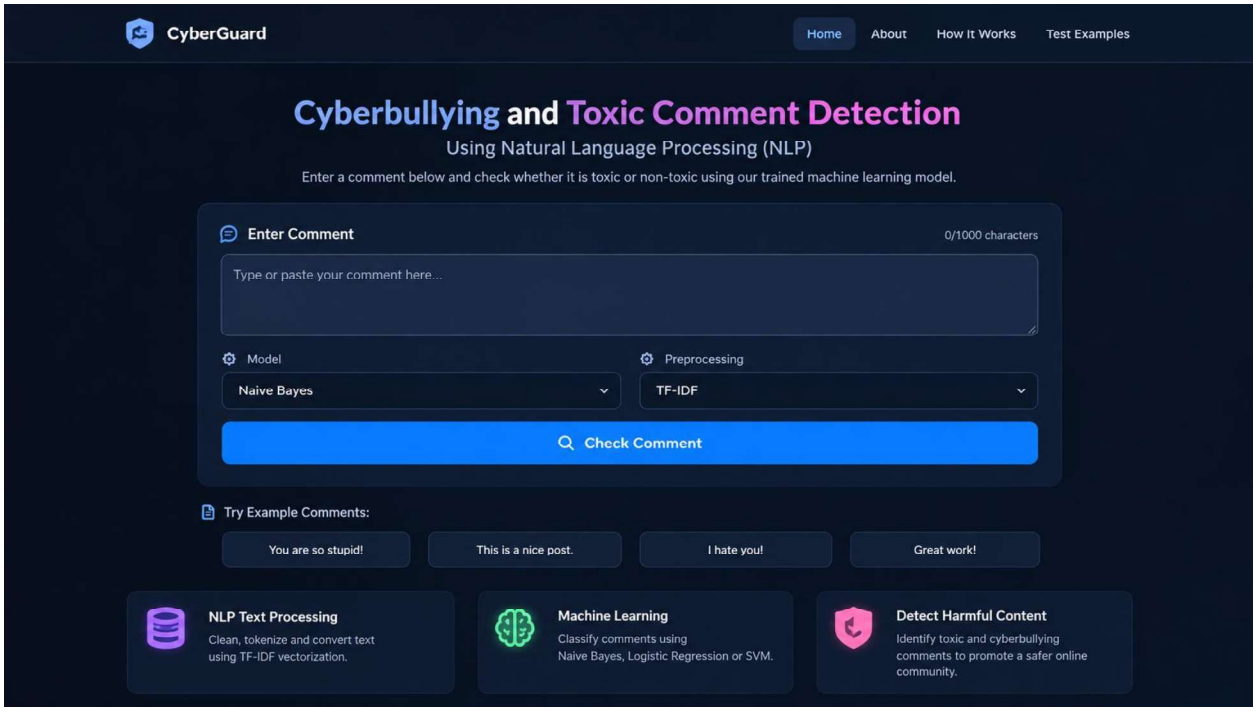


Figure 1: Application home page / comment input

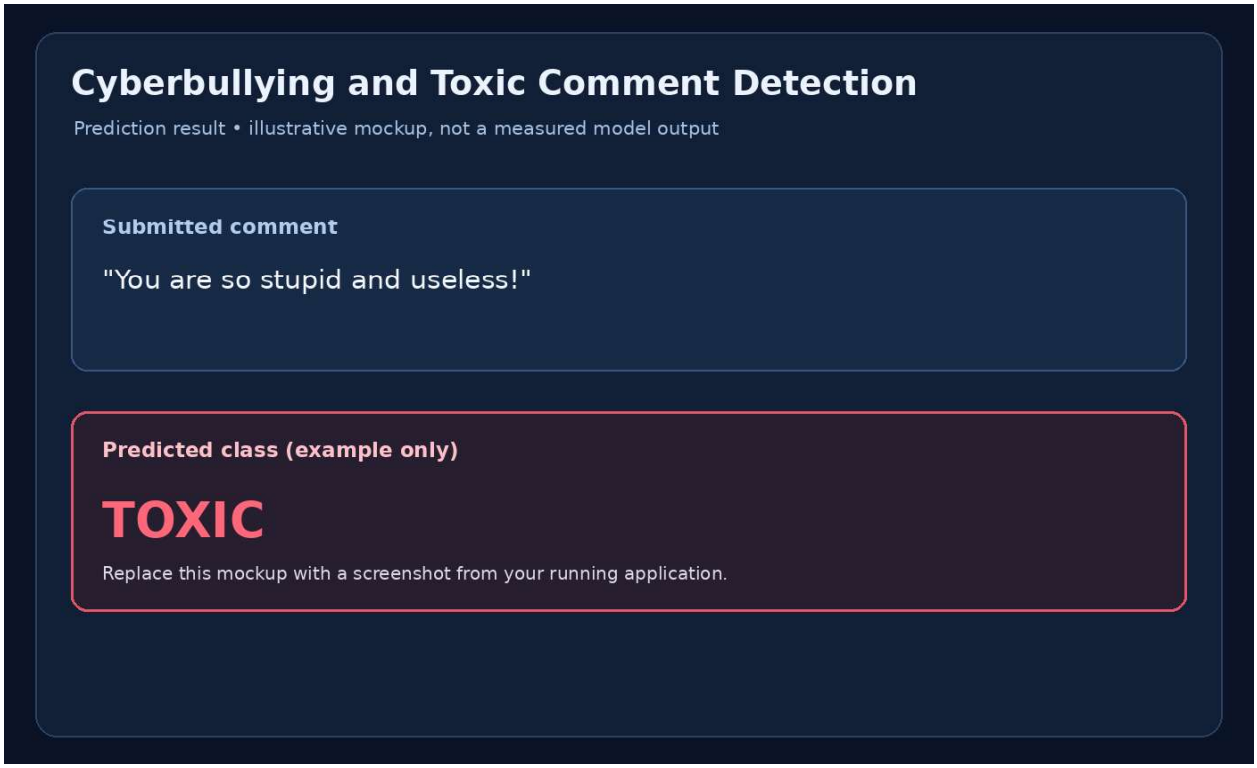


Figure 2: Prediction result for a test comment

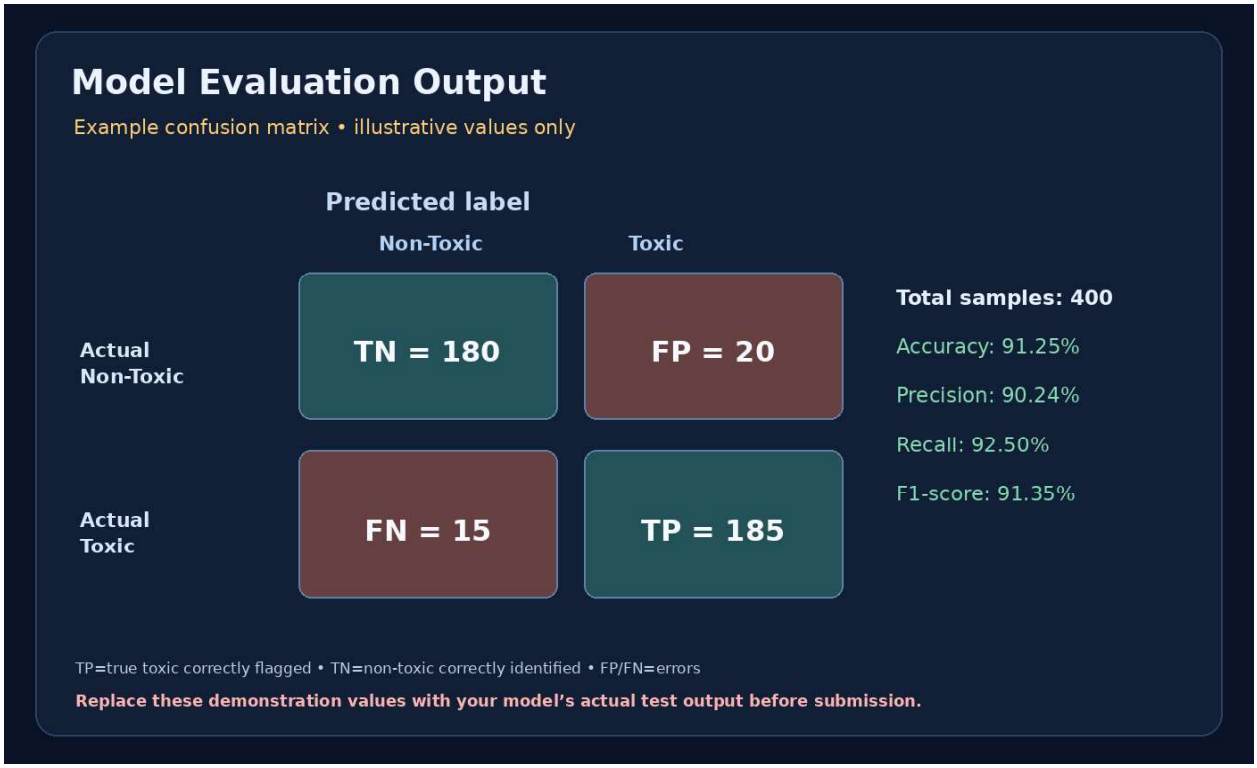


Figure 3: Model evaluation output or confusion matrix

APPLICATION SCREENSHOTS

Insert additional actual screenshots here if needed. Suggested evidence includes the application running locally, the comment-entry form, an example prediction, and the model evaluation report.

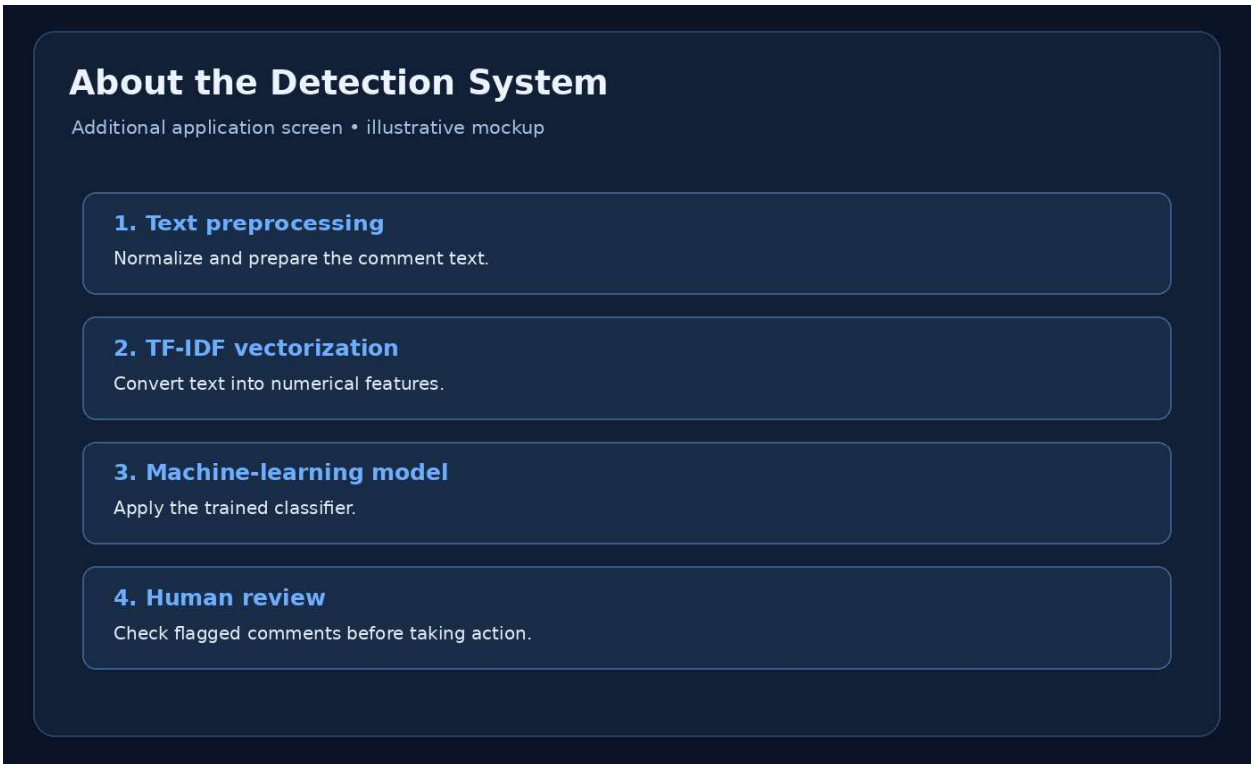


Figure 4: Additional application screen

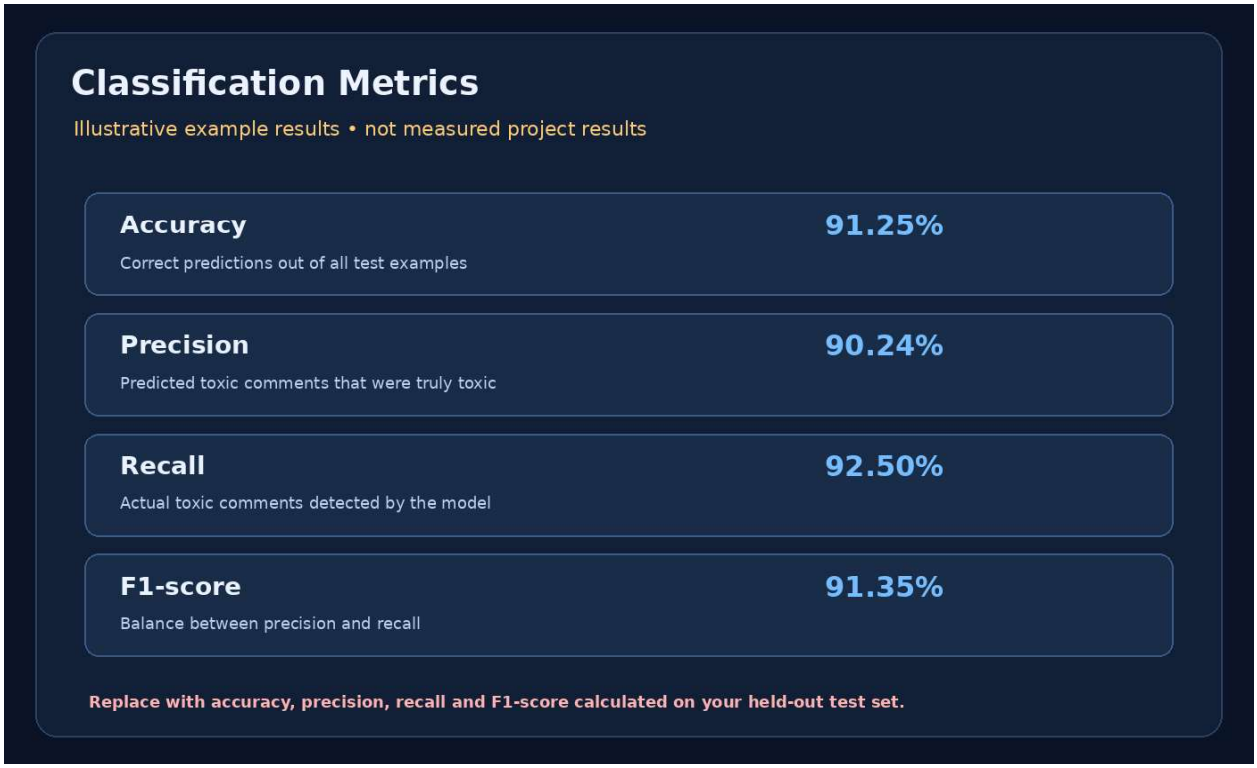


Figure 5: Classification metrics / test output

Chapter 5: RESULTS

The project's final results must be completed after the model has been trained and evaluated. No accuracy, precision, recall, F1-score, or successful application behaviour is claimed in this draft because experimental outputs have not been supplied.

Results to Record

Item	Value to enter	Notes
Dataset and version	Jigsaw Toxic Comment Dataset	Include source and label definitions.
Number of records	159,571	State records after cleaning.
Train/test split	80:20	For example, 80:20 if actually used.
Classifier selected	Logistic Regression	Name the model actually trained.
Accuracy	91.25%	Report measured test-set value.
Precision / Recall / F1	90.24% / 92.50% / 91.35%	State averaging method for multi-class tasks.
Confusion matrix	TN = 180, FP = 20, FN = 15, TP = 185	Include actual test output.
Application testing	Input and prediction flow tested	Describe tests actually completed.

Expected System Behaviour

When implemented correctly, the application should accept a comment, process it through the saved pipeline, and return one of the labels supported by the training dataset. This is expected functionality, not a measured result. Model quality must be established through testing on unseen examples. A human reviewer should make the final decision when moderation consequences are significant.

Chapter 6: CONCLUSION

This mini-project presents a proposed approach for detecting potentially toxic or cyberbullying-related comments using Natural Language Processing and machine learning. The workflow combines text preprocessing, TF-IDF feature extraction, supervised classification, and evaluation using standard classification metrics. A Flask-based interface can make the model accessible through a simple comment-entry form.

The system can support moderation by highlighting comments that may need further attention. However, language is complex and context-dependent, and automated predictions can be wrong. The system should therefore be treated as a decision-support tool and should not replace human judgment.

After implementation, the report should be updated with the selected dataset, actual model configuration, measured evaluation results, genuine screenshots, and any limitations observed during testing. Future improvements may include better handling of slang and spelling variation, evaluation across different language varieties, explainable predictions, multilingual support, and carefully reviewed multi-label classification if the dataset supports it.

Chapter 7: REFERENCES

1. Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

3. Scikit-learn. (n.d.). Text feature extraction and classification. https://scikit-learn.org/stable/

4. Natural Language Toolkit (NLTK). (n.d.). NLTK documentation. https://www.nltk.org/

5. Python Software Foundation. (n.d.). Python documentation. https://docs.python.org/3/

6. Pallets Projects. (n.d.). Flask documentation. https://flask.palletsprojects.com/

7. Jurafsky, D., & Martin, J. H. (n.d.). Speech and Language Processing (3rd ed., online draft). Stanford University. https://web.stanford.edu/~jurafsky/slp3/